What the model learned during training

Before inference, understand what each loss contributed:

- **Cross-entropy** trained the classifier head to map embeddings → class logits. It learned a decision boundary per class.
- **Triplet loss** trained the embedding space to be geometrically meaningful — same make/model clusters together, different ones are pushed apart.

Both paths share the **same backbone + embedding head**. Only the final step differs.

The Embedding Journey (shared for both paths)

```
Input image (H × W × 3)
        │
        ▼
   ┌──────────────┐
   │  Backbone    │    ResNet50 / DINOv2
   │  (frozen or  │    Extracts spatial features
   │   fine-tuned)│     Output: feature vector (2048-d or 768-d)
   └──────────────┘
        │
        ▼
   ┌──────────────┐
   │  Embedding   │    Linear → BN → ReLU → Dropout
   │    Head      │    Output: 512-d embedding vector
   └──────────────┘
        │
        ▼
   ┌──────────────┐
   │  L2 Norm     │    ‖e‖ = 1  (unit hypersphere)
   └──────────────┘
        │
    embedding e          ← this is the core representation
    (512-d, unit norm)
        │
    ┌───┴───┐
    ▼       ▼
Path A      Path B
Classifier  Gallery
  Head      Search
```

Path A — Closed-set Classification (cross-entropy path)

Use this when: your make/model classes are **fixed and known** at deployment time.

```
embedding e  (512-d)
      │
      ▼
┌──────────┐
│  Linear  │   W ∈ ℝ^(num_classes × 512)
│ Classifier│   Each row = learned weight vector for one
class
└──────────┘
      │
      ▼
  logits z   (num_classes-d)   raw unnormalised scores
      │
      ▼
  softmax(z)                   probabilities per class
      │
      ▼
  top-k classes + confidence
```

What's actually happening geometrically:

Each row of the classifier weight matrix W is a **prototype vector** for that class. The dot product `W[i] · e` measures alignment between the embedding and the class prototype. Softmax normalises these into probabilities.

This is why ArcFace improves over plain cross-entropy — it enforces that the angle between `e` and `W[i]` must be meaningfully smaller for the correct class, not just the dot product.

```
@torch.no_grad()
def classify(model, image):
    # Step 1 — backbone
    features = model.backbone(image)          # (1, 2048)

    # Step 2 — embedding head
    embedding = model.embedding_head(features) # (1, 512)

    # Step 3 — L2 normalise
    embedding = F.normalize(embedding, dim=1)  # (1, 512),
unit norm

    # Step 4 — classifier
    logits = model.classifier(embedding)       # (1,
num_classes)

    # Step 5 — probabilities
    probs = torch.softmax(logits, dim=1)       # (1,
num_classes)
```

```
    # Step 6 — decision
    top_probs, top_indices = probs.topk(3)
    return top_indices, top_probs
```

Confidence interpretation:

Softmax output	Meaning
> 0.90	High confidence — clean image, seen class
0.60 – 0.90	Moderate — partial occlusion, similar models
0.40 – 0.60	Uncertain — viewpoint never seen, similar classes competing
< 0.40	Reject / flag for review — likely unseen class or bad crop

Path B — Open-set Gallery Search (triplet path)

Use this when: you want to handle **new makes/models not in training**, or do cross-camera vehicle matching.

```
embedding e  (512-d, L2 normalised)
      │
      ▼
  pgvector / FAISS gallery
  (pre-built index of labelled embeddings)
      │
      ▼  cosine similarity search
      │
  top-k nearest neighbours
      │
      ▼
  majority vote or
  weighted average of neighbour labels
      │
      ▼
  predicted make/model + similarity score
@torch.no_grad()
def gallery_search(model, image, gallery_index,
gallery_labels, top_k=5):
    # Get embedding — same first 3 steps as Path A
    features  = model.backbone(image)
    embedding = model.embedding_head(features)
    embedding = F.normalize(embedding, dim=1)   # (1, 512)

    query = embedding.cpu().numpy()
```

```
    # Search gallery — cosine similarity
    # Since embeddings are L2-normalised, cosine sim = dot
product
    similarities, indices = gallery_index.search(query,
top_k)

    # Retrieve labels of nearest neighbours
    neighbour_labels = [gallery_labels[i] for i in
indices[0]]
    neighbour_sims   = similarities[0]

    # Weighted majority vote
    votes = {}
    for label, sim in zip(neighbour_labels, neighbour_sims):
        votes[label] = votes.get(label, 0) + sim

    predicted = max(votes, key=votes.get)
    confidence = votes[predicted] / sum(votes.values())

    return predicted, confidence
```

Which path to use and when

Scenario	Path	Reason
Fixed fleet (known vehicle types)	A — classifier	Fast, direct, no gallery needed
New models added frequently	B — gallery	Add new embeddings without retraining
Cross-camera vehicle re-identification	B — gallery	Same logic as ReID
Confidence + reject option needed	A with threshold	Softmax confidence easy to threshold
Few labelled examples per class	B — gallery	1-shot / few-shot capable
DeepStream secondary GIE	A — classifier	TensorRT-friendly, single forward pass

The subtle interplay between the two losses at inference

This is the key insight — **triplet loss made Path A better**, even though triplet doesn't touch the classifier:

```
During training:

  Triplet loss acts here:            CE / ArcFace acts here:

  ┌──────────────────────┐           ┌──────────────────────┐
  │  Embedding space     │           │  Classifier weights  │
```

```
┌──────────────────────┐   ──────→   ┌──────────────────────────┐
│                      │             │                          │
│  •• Toyota Camry     │  tighter    │  W_camry  ──────→  ●     │
│                      │  clusters   │                          │
│  ○○ Honda Accord     │             │  W_accord ──────→  ○     │
└──────────────────────┘             └──────────────────────────┘
```

```
Triplet forces same-class embeddings to cluster tightly.
This means the dot product W[correct] · e is more
consistently high.
Result: softmax peaks are sharper → higher confidence on
correct class.
```

So at inference, even on Path A:

- The embedding is more **discriminative** because of triplet
- The classifier sees a cleaner, more separable input
- Confidence scores are more **calibrated** — high confidence actually means correct

And on Path B, the gallery search works well precisely because triplet directly optimised the embedding geometry you're searching over.

Common inference failure modes specific to VMMR

1. Crop quality dependency — both paths assume a clean vehicle crop. A bad detection bbox (partial vehicle, background included) will produce a poor embedding. Always validate IoU of detection bbox before feeding to VMMR.

2. Viewpoint collapse — if training data had mostly front/rear views, the embedding space didn't learn side-view representations. The softmax will still output something, but confidence will be low and often wrong. TTA helps here.

3. Intra-class year variation — a 2015 and 2022 Civic have different front fascias. If your classes are make/model only (not year), the embedding is trained to ignore year differences. This is correct behaviour but can confuse users who expect year-level prediction.

4. Softmax overconfidence — cross-entropy training is known to produce overconfident predictions on out-of-distribution inputs. A completely unseen vehicle type will still get a confident-looking prediction. Mitigation: combine softmax confidence with embedding distance to nearest gallery neighbour. If both are high → trust it. If softmax is high but gallery distance is large → flag it.

VMMR vs ReID — the key distinction

	VMMR	ReID
Goal	"What make/model is this?"	"Have I seen this specific vehicle before?"
Classes	Fixed (Toyota Camry, Honda Civic…)	Open — new identities appear constantly
Classifier head at inference	✅ Used (Path A)	❌ Discarded always
Embedding search	Optional (Path B)	Mandatory — the only path
Query	Single image	Image vs entire gallery

In ReID, the classifier head is thrown away at inference entirely. Cross-entropy was only used during training to give the backbone a strong supervisory signal. It never runs at inference.

What each loss built during training

```
Cross-entropy / ArcFace
    → Forced the backbone to learn identity-discriminative
features
    → Each training ID got a tight cluster in embedding space
    → The classifier weight W[i] became a prototype for
identity i
    → Side effect: embedding space is well-organised even for
unseen IDs

Triplet loss
    → Directly optimised inter-identity distances
    → Same identity from Camera A and Camera B → close in
embedding space
    → Different identities that look similar → pushed apart
    → This generalises to identities never seen during
training
```

Both losses shaped the **same embedding space**. The classifier head just gets detached at inference.

The Full Inference Pipeline

```
Query image (vehicle crop from camera)
        │
        ▼
┌──────────────────┐
│   Backbone       │   ResNet50 / DINOv2
│                  │   Learns appearance features:
│                  │   colour, shape, texture, parts
│                  │   Output: 2048-d (ResNet) or 768-d (DINOv2)
```

```
└──────────────┘
       │
       ▼
┌──────────────┐   Linear → BN → ReLU
│  Embedding   │   Output: 256-d (resnet50_market1501
│    Head      │
standard)
└──────────────┘
       │
       ▼
┌──────────────┐   Projects onto unit hypersphere
│   L2 Norm    │   All embeddings have ‖e‖ = 1
│              │   Cosine similarity = dot product now
└──────────────┘
       │
       ▼
  query embedding q  (256-d, unit norm)
       │
       ▼
┌──────────────────────────┐
  pgvector / FAISS  ◄──────────────────│  Gallery
│
  similarity search                    │  Pre-built index of
│
       │                               │  known vehicle
embeddings│
       ▼                               │  each entry: (id,
emb)  │
  top-k nearest neighbours
└──────────────────────────┘
       │
       ▼
  Decision logic
  (threshold / re-rank / fuse)
       │
       ▼
  "Known vehicle — ID_042"  OR  "New vehicle — add to
gallery"
```

Step-by-step in code

```
@torch.no_grad()
def reid_inference(model, query_image, gallery,
threshold=0.75):
```

```
    # ── Step 1: Backbone forward pass ──────────────────────────────
    features = model.backbone(query_image)      # (1, 2048)
    # Backbone learned: colour histogram, shape contours,
    # part-level features (wheels, roof, windows)
    # DINOv2 also captures fine texture via attention

    # ── Step 2: Embedding projection ───────────────────────────────
    embedding = model.embedding_head(features)  # (1, 256)
    # Compresses 2048-d into compact 256-d space
    # BN makes it robust to batch statistics shift

    # ── Step 3: L2 normalisation ───────────────────────────────────
    q = F.normalize(embedding, dim=1)           # (1, 256), ‖q‖ = 1
    # Now cosine_sim(q, g) = dot(q, g) for any gallery vector g
    # Triplet loss trained this space:
    #   same vehicle from diff cameras → cosine sim > 0.8
    #   diff vehicles                  → cosine sim < 0.5

    # ── Step 4: Gallery search ─────────────────────────────────────
    q_np = q.cpu().numpy()
    similarities, indices = gallery.index.search(q_np, k=10)
    # Returns 10 nearest neighbours by cosine similarity
    # Complexity: O(log N) with HNSW index in pgvector

    # ── Step 5: Decision ───────────────────────────────────────────
    best_sim   = similarities[0][0]
    best_id    = gallery.ids[indices[0][0]]

    if best_sim >= threshold:
        return {"status": "known", "id": best_id, "similarity": best_sim}
    else:
        new_id = gallery.insert(q_np)
        return {"status": "new",   "id": new_id, "similarity": best_sim}
```

The threshold decision in depth

This is where most real-world ReID systems fail or succeed. The similarity score between query and gallery means:

```
Cosine similarity

  1.0  ─── identical (same image)
  0.9  ─── same vehicle, similar viewpoint
  0.8  ─── same vehicle, different camera / lighting     ←
target operating point
  0.7  ─── ambiguous zone: different vehicle, similar
appearance
  0.6  ─── different vehicles, same make/model
  0.5  ─── random similarity
  0.0  ─── orthogonal (completely different)
# Don't use a single global threshold
# Use per-class adaptive thresholds based on
# intra-class variance learned during training

def adaptive_threshold(best_sim, second_sim, top_k_labels):
    # Ratio test (like SIFT matching)
    # If best match is significantly better than second-best
→ confident
    ratio = best_sim / (second_sim + 1e-6)
    if ratio > 1.3 and best_sim > 0.70:
        return "known", "high_confidence"

    # Consistency check across top-k
    top_label = top_k_labels[0]
    agreement = sum(1 for l in top_k_labels if l ==
top_label) / len(top_k_labels)
    if agreement >= 0.6 and best_sim > 0.65:
        return "known", "medium_confidence"

    return "unknown", "low_confidence"
```

Re-ranking (production quality boost)

Raw nearest neighbour search is not the best you can do. Re-ranking uses the intuition: **if A is truly the same as B, then B's neighbours should also contain A**.

```
Query q finds gallery entries: [G1, G2, G3, G4, G5]

Re-ranking check:
  Take G1. Search gallery using G1 as query.
  Do G1's neighbours include q or embeddings close to q?
  If yes → G1 is a strong match (mutual nearest neighbour)
```

```
  If no  → G1 might be a false match

Mutual nearest neighbours → boost their similarity score
One-way nearest neighbours → penalise their score
def rerank(query_emb, gallery_embs, gallery_ids, k1=20, k2=6,
lambda_val=0.3):
    """
    k-reciprocal re-ranking (Zhong et al. 2017)
    Standard in top ReID systems
    """
    all_embs = np.vstack([query_emb, gallery_embs])  # (N+1,
256)
    n = len(all_embs)

    # Pairwise distance matrix
    dist_mat = 1 - np.dot(all_embs, all_embs.T)  # cosine
distance

    # k-reciprocal neighbours for each sample
    def get_reciprocal(idx, k):
        # Forward k-nearest
        forward  = np.argsort(dist_mat[idx])[:k+1]
        # Keep only those who also have idx in their k-
nearest (mutual)
        reciprocal = [j for j in forward
                      if idx in np.argsort(dist_mat[j])
[:k+1]]
        return set(reciprocal)

    # Jaccard distance replaces original distance
    # Two samples are similar if they share many reciprocal
neighbours
    query_reciprocal = get_reciprocal(0, k1)

    reranked_scores = []
    for i in range(1, n):
        gallery_reciprocal = get_reciprocal(i, k1)
        # Jaccard similarity
        intersection = len(query_reciprocal &
gallery_reciprocal)
        union        = len(query_reciprocal |
gallery_reciprocal)
        jaccard_sim  = intersection / (union + 1e-6)

        # Blend original distance with Jaccard
```

```
        original_dist = dist_mat[0][i]
        final_dist    = (1 - lambda_val) * (1 - jaccard_sim)
+ lambda_val * original_dist
        reranked_scores.append((gallery_ids[i-1],
final_dist))

    return sorted(reranked_scores, key=lambda x: x[1])
```

Gallery management at inference time

This is unique to ReID vs VMMR — the gallery is **live and growing**:

```
class ReIDGallery:
    def __init__(self, sim_threshold=0.80,
max_embeddings_per_id=10):
        self.index    = faiss.IndexFlatIP(256)   # Inner
product = cosine (unit norm)
        self.ids      = []
        self.counts   = {}
        self.threshold = sim_threshold
        self.max_emb   = max_embeddings_per_id

    def query_or_register(self, embedding: np.ndarray) ->
str:
        if self.index.ntotal == 0:
            return self._register_new(embedding)

        sims, idxs =
self.index.search(embedding.reshape(1,-1), k=5)
        best_sim  = sims[0][0]
        best_id   = self.ids[idxs[0][0]]

        if best_sim >= self.threshold:
            # Known vehicle — optionally update gallery
embedding
            self._update_gallery(best_id, embedding)
            return best_id
        else:
            return self._register_new(embedding)

    def _update_gallery(self, vehicle_id, new_embedding):
        # Running average — makes gallery more robust over
time
        # Don't store too many per ID (memory)
        count = self.counts.get(vehicle_id, 1)
```

```
        if count < self.max_emb:
            self.index.add(new_embedding.reshape(1,-1))
            self.ids.append(vehicle_id)
            self.counts[vehicle_id] = count + 1

    def _register_new(self, embedding) -> str:
        new_id = f"VEH_{self.index.ntotal:05d}"
        self.index.add(embedding.reshape(1,-1))
        self.ids.append(new_id)
        self.counts[new_id] = 1
        return new_id
```

Why cross-entropy helped even though it's discarded

This is the non-obvious but important part:

```
During training, CE loss on N identity classes:

  Each class gets a weight vector W[i] in the classifier
  Gradient flows back through embedding head into backbone
  Backbone is forced to produce features where W[i]·e is
  maximised for the correct class

  Side effect:
  The embedding space becomes structured even BETWEEN
  training identities — the geometry generalises

  Analogy:
  If you train on 10,000 vehicle IDs with CE + triplet,
  the model learns "what makes vehicles distinguishable"
  as a general concept, not just memorising 10,000 IDs

  At inference on a never-seen vehicle:
  The backbone already knows to look at headlights,
  grilles, body lines — CE taught it this implicitly
```

This is why training on large identity datasets (Market-1501 with 1,501 IDs, VeRi-776 with 776 vehicle IDs) generalises well to your specific deployment camera setup — even if none of your actual vehicles were in the training set.

The complete picture in one diagram

```
TRAINING                                INFERENCE
────────                                ─────────

Image + Identity label                  Query image (no label)
```

```
        │                                   │
        ▼                                   ▼
    Backbone  ◄── gradient from CE        Backbone
        │         and Triplet               │
        ▼                                   ▼
  Embedding head ◄── both losses        Embedding head
        │                                   │
        ▼                                   ▼
   256-d vector                          256-d vector
        │                                   │
   ┌────┴────┐                           L2 normalise
   │         │                             │
   ▼         ▼                             ▼
Classifier  L2 norm                    Gallery search
  head        │                             │
   │      Triplet loss                  Threshold +
CE loss    operates                    re-ranking
   │       here                            │
   └────────┘                              ▼
   Both update                         Known / New ID
   same backbone
```

The classifier head at inference is like scaffolding — essential during construction (training) to shape the structure (embedding space), then removed when the building (inference system) stands on its own.

FAILURE MODES

Great question — knowing failure modes is what separates production systems from research demos.

Shared Failure Modes (both ReID and VMMR)

1. Poor Detection Crop Quality

The embedding model only sees the **cropped bbox** from your detector. Garbage in, garbage out.

```
Bad crop scenarios:

   ┌──────────────────┐     ┌──────────────────┐
 ┌─────────────────────┐
   │  ████            │     │  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒  │     │
 ██████████      │
   │  ██CAR▒  [truck] │     │  ▒background▒    │     │
partially         │
```

```
  │ ████            │     │ ▒▒in crop▒▒▒    │     │ occluded
│
  └─────────────────┘     └─────────────────┘
└─────────────────┘

  Overlapping detections   Large bbox padding       Occlusion
cuts
  → embedding mixes two    → background dominates   → missing
features
    vehicles' features       the embedding
backbone relies on
```

Effect on ReID: Two vehicles get similar embeddings → false match **Effect on VMMR:** Classifier sees wrong context → wrong make/model

Mitigation:

```
def validate_crop(bbox, image_shape, min_size=64,
max_aspect=4.0):
    x1, y1, x2, y2 = bbox
    w, h = x2 - x1, y2 - y1
    img_h, img_w = image_shape[:2]

    # Too small — not enough pixels for features
    if w < min_size or h < min_size:
        return False, "too_small"

    # Weird aspect ratio — likely bad detection
    aspect = max(w, h) / (min(w, h) + 1e-6)
    if aspect > max_aspect:
        return False, "bad_aspect"

    # Too much outside frame — partial vehicle
    overlap = (min(x2, img_w) - max(x1, 0)) * (min(y2, img_h)
- max(y1, 0))
    crop_area = w * h
    if overlap / crop_area < 0.85:
        return False, "out_of_frame"

    return True, "ok"
```

2. Illumination & Appearance Shift

The same vehicle looks dramatically different across:

- Day vs night (IR cameras flip colour entirely)
- Tunnel entrance/exit (sudden brightness change)
- Wet vs dry (reflections alter colour histogram)
- Headlights on vs off

```
Same vehicle:
  Daytime embedding:   [0.82, -0.31, 0.45, ...]
  Night IR embedding:  [0.21,  0.67, -0.53, ...]
  Cosine similarity:   0.41  ← below threshold → treated as
new vehicle
```

Mitigation:

- Train with aggressive photometric augmentation (RandomShadow, CLAHE, channel shuffle)
- Normalise crops with histogram equalisation before feeding to model
- Maintain separate day/night gallery embeddings per identity
- Use temporal smoothing — don't update gallery from single-frame anomalies

3. Intra-class Visual Similarity

Some classes are genuinely hard to distinguish:

VMMR:

```
2019 Honda Civic  vs  2019 Honda Accord  → very similar front
fascia
Toyota Camry      vs  Toyota Avalon      → near-identical
body lines
White sedan A     vs  White sedan B      → colour + shape
both match
```

ReID:

```
Two black SUVs of same make/model
→ their embeddings are naturally close
→ threshold that works for distinct vehicles
  causes false matches here
```

Mitigation: Hierarchical classification (make → model → year) — coarser levels are easier and constrain the harder fine-grained prediction.

4. Domain Gap (Training vs Deployment)

Models trained on benchmark datasets fail silently on your specific cameras.

```
Training data (VeRi-776):           Your deployment:
  - Daylight, clean images            - Low-res CCTV, 15fps
  - Multiple viewpoints balanced      - Mostly rear/side
views
  - Chinese urban traffic             - Your city, different
vehicles
  - Clean bboxes                      - Overlapping traffic
```

Mitigation:

- Fine-tune on even a small amount of labelled data from your cameras

- Domain adaptation — use unlabelled footage from deployment cameras with self-supervised pretraining
- Monitor embedding distribution shift in production (flag when query embeddings fall outside training distribution)

ReID-Specific Failure Modes

5. ID Fragmentation (already discussed, but the embedding angle)

Occlusion causes the tracker to lose a vehicle and assign a new track ID. But even if the tracker recovers, the ReID system may fail to merge them:

```
t=0:   Vehicle appears     → track_001, embedding e1 →
gallery entry G1
t=45:  Occluded by truck   → track lost
t=52:  Reappears           → track_002, embedding e2
       ReID query: sim(e2, G1) = 0.71  ← just below threshold
0.75
       → Registered as NEW vehicle G2

Root cause:
  During occlusion, vehicle moved + viewpoint changed
slightly
  e2 is a valid embedding but just far enough from e1
  to fall below threshold
```

Mitigation: Lower threshold for re-entry in known occlusion zones. Use motion prediction (Kalman filter) to weight spatial proximity alongside embedding similarity.

6. Gallery Contamination Over Time

The gallery accumulates errors. One bad embedding (motion blur, partial occlusion during insertion) becomes a **permanent bad reference**:

```
# Problem: naive gallery update
def bad_update(gallery, new_embedding, matched_id):
    gallery[matched_id].append(new_embedding)   # blindly
adds everything
    # After 100 frames: gallery has 20 good embeddings
    #                   and 80 corrupted embeddings
    # Mean embedding drifts away from true appearance
# Better: quality-gated update
def quality_gated_update(gallery, new_embedding, matched_id,
quality_score):
    # Only update if this embedding is high quality
    # Quality = confidence from detector × crop completeness
    if quality_score > 0.85:
```

```
        # Running weighted average — recent observations
weighted higher
        alpha = 0.1
        gallery[matched_id] = (
            (1 - alpha) * gallery[matched_id] +
            alpha * new_embedding
        )
```

7. Threshold Sensitivity

A single global threshold fails in practice because:

```
Scenario A — easy case:
  Query: clear frontal view, good lighting
  Best match similarity: 0.91 → correctly identified

Scenario B — hard case:
  Query: rear view, night camera, same vehicle
  Best match similarity: 0.72 → SAME threshold → missed!

Scenario C — false positive risk:
  Query: different white SUV, same make/model
  Best match similarity: 0.76 → wrong match!

The threshold that catches scenario B will cause false
positives in C.
These are fundamentally in tension.
```

Mitigation:

```
def contextual_threshold(query_emb, top_k_sims, camera_id,
time_of_day):
    base = 0.75

    # Night cameras have noisier embeddings → relax threshold
    if time_of_day == 'night':
        base -= 0.05

    # If top-2 similarities are very close → ambiguous →
tighten
    if len(top_k_sims) >= 2:
        gap = top_k_sims[0] - top_k_sims[1]
        if gap < 0.05:
            base += 0.08   # require more separation to
commit

    # Camera-specific calibration learned from validation set
```

```
    base += camera_calibration[camera_id]

    return base
```

8. Cross-Camera Viewpoint Mismatch

Camera A sees the front of a vehicle. Camera B sees the rear. The model must match these despite radically different appearances:

```
Camera A (entrance):   front view  → embedding_front
Camera B (exit):       rear view   → embedding_rear

sim(embedding_front, embedding_rear) is inherently lower
than sim(embedding_front, embedding_front)

If threshold is calibrated on same-camera pairs → too tight
for cross-camera
If threshold is loosened for cross-camera → false positives
same-camera
```

Mitigation: Train with viewpoint-diverse triplets — ensure each triplet has anchor and positive from **different viewpoints**. Models trained this way learn viewpoint-invariant features instead of viewpoint-specific ones.

VMMR-Specific Failure Modes

9. Softmax Overconfidence on Unknown Classes

Cross-entropy training produces overconfident predictions on out-of-distribution inputs. A vehicle make/model that was never in training data will still get a high-confidence wrong prediction:

```
# The model always outputs something — it doesn't know what
it doesn't know
logits  = [2.1, 0.3, -0.5, ...]   # for unseen Rivian R1T
truck
probs   = [0.89, 0.08, 0.02, ...]  # 89% "Toyota Tundra" —
confidently wrong

# Detection: combine classifier confidence with embedding
distance
def reliable_prediction(logits, embedding, gallery_index,
threshold=0.70):
    probs       = torch.softmax(logits, dim=1)
    cls_conf    = probs.max().item()
    pred_class  = probs.argmax().item()
```

```
    # Also check: does this embedding actually fit the
predicted class gallery?
    class_gallery_embs =
gallery_index.get_class_embeddings(pred_class)
    gallery_sim = cosine_similarity(embedding,
class_gallery_embs.mean(0))

    # Trust prediction only if BOTH are confident
    if cls_conf > 0.80 and gallery_sim > threshold:
        return pred_class, cls_conf, "confident"
    else:
        return pred_class, cls_conf, "uncertain — flag for
review"
```

10. Long-tail Class Starvation

Your loss is dominated by frequent classes. Rare classes get almost no gradient signal:

```
Training dataset distribution:
  Toyota Camry:     8,432 images  → loss contribution: high
  Honda Civic:      7,891 images  → loss contribution: high
  ...
  Pagani Zonda:        12 images  → loss contribution: ~0
  Morgan Three-Wheeler: 8 images  → almost never sampled

Result:
  Pagani Zonda embedding is undertrained
  → collapses near its nearest frequent class
  → always misclassified as something common
```

Mitigation:

```
# Class-balanced sampling ensures every class gets equal
gradient
from pytorch_metric_learning.samplers import MPerClassSampler

sampler = MPerClassSampler(
    labels=train_labels,
    m=4,                  # exactly 4 samples per class per
batch
    batch_size=128,
    length_before_new_iter=len(train_dataset)
)

# Also: use class-weighted loss
class_counts  = np.bincount(train_labels)
```

```
class_weights = 1.0 / np.log1p(class_counts)   # log
smoothing
class_weights = torch.FloatTensor(class_weights).cuda()
ce_loss = nn.CrossEntropyLoss(weight=class_weights)
```

11. Year-Level Confusion

Adjacent model years are nearly identical visually but have separate class labels:

```
2021 Toyota RAV4  ──→  class 1847
2022 Toyota RAV4  ──→  class 1848

Visually: nearly identical (minor grille update)
Embedding: very close but forced to be separate classes
Triplet loss: pushes them apart → fights against visual
reality
CE loss: penalises confusion between them equally as
confusing Civic with RAV4

Result:
  Model becomes uncertain on year boundaries
  High entropy in softmax between adjacent years
  Accuracy metric looks bad even when prediction is "almost
right"
```

Mitigation: Hierarchical label smoothing — apply more smoothing between visually-adjacent classes (same model, adjacent years) than between distinct models.

Summary: Failure Mode Map

```
Input quality          Embedding quality       Decision
quality
____________           _________________
________________

Bad crop          →    Corrupted embedding  →  Wrong match
Occlusion         →    Partial features     →  ID
fragmentation (ReID)
Illumination      →    Distribution shift   →  Miss known
vehicle
Domain gap        →    OOD embeddings       →  Systematic
errors

                       Gallery drift        →  Degradation
over time (ReID)
```

```
                        Long-tail starvation →  Rare class
failure (VMMR)
                        Year confusion       →  Adjacent class
errors (VMMR)
                        Overconfidence       →  Silent wrong
prediction (VMMR)

                                                Threshold too
tight → misses
                                                Threshold too
loose → false pos
                                                Cross-camera
gap    → misses
```

The most dangerous are the **silent failures** — overconfident wrong predictions (VMMR) and gallery drift (ReID) — because they don't surface as errors in logs, they just quietly corrupt your downstream analytics.